

Deploy and security

Put your project online for free, and keep your keys safe while you do it.

github.com/murthyroshan/innoblock-2.0-starter

Deployment

The whole hackathon costs ₹0: the backend on Render, the frontend on Vercel or Netlify, records in Neon or Supabase, and UptimeRobot to keep the backend awake. All of them give you HTTPS automatically.

Deploy in this order: database, backend, frontend, keep-alive.

1 · Database: Neon or Supabase

Locally the backend saves records in a SQLite file. On Render that file is wiped whenever the service restarts, so deployed records need Postgres. The backend switches to Postgres by itself when `DATABASE_URL` is set: no code changes.

- **Neon:** create a project → **Connect** → copy the connection string.
- **Supabase:** create a project → **Connect** → copy the **Session pooler** string (the direct one doesn't work from Render), and put in your database password.

Test it locally first: paste the string as `DATABASE_URL` in `backend/.env`, restart `python app.py`, store a record, and check it appears in the table in the Neon or Supabase dashboard.

2 · Backend on Render

1. Push your repo to GitHub (check `git status` first: `.env` must not be listed).
2. Render → **New** → **Web Service** → connect your repo.
3. Root Directory: `backend` · Runtime: **Python**.
4. Build command: `pip install -r requirements.txt`
5. Start command: `gunicorn app:app --timeout 120` (the timeout leaves room to wait for a transaction to be mined).
6. Instance type: **Free**.
7. Environment: add every variable from your `backend/.env`, including `DATABASE_URL`. Leave `FRONTEND_ORIGIN` as `*` until step 3.

Done when `https://<your-service>.onrender.com/health` shows `"ok": true` and your network's `chainId`.

3 · Frontend on Vercel or Netlify

1. In `frontend/config.js`, set `BACKEND_URL` to your Render URL (`https://...onrender.com`, no trailing slash). Commit and push.
2. **Vercel:** Add New → **Project** → import the repo → Root Directory: `frontend` → Framework preset: **Other** → no build command → Deploy.

3. **Or Netlify Drop:** drag the `frontend` folder onto app.netlify.com/drop. Live in seconds, and the fastest fallback if anything else fails.
4. Back on Render, set `FRONTEND_ORIGIN` to the exact frontend URL (`https://your-app.vercel.app` , no trailing slash). Render redeploys.

Done when the full flow works on the live URL from a teammate's phone, inside the MetaMask app's browser.

4 · Keep the backend awake

1. [UptimeRobot](#) → **Add New Monitor** → type **HTTP(s)**.
2. URL: `https://<your-service>.onrender.com/health`
3. Interval: **5 minutes**.

Free-tier gotchas

Gotcha	What it means for you
Free backends sleep	Render spins a free service down after 15 minutes without traffic, and the next request takes about a minute . UptimeRobot prevents it; open <code>/health</code> yourself just before you demo.
Render's disk is wiped on restart	Anything the backend writes to disk is lost. That's why records go in Neon or Supabase.
Render's own free Postgres expires	It's deleted 30 days after creation. Use Neon or Supabase instead.
750 free hours a month	Per Render workspace. One service running all month fits; several copies don't.
CORS is exact	<code>FRONTEND_ORIGIN</code> must match the frontend URL exactly: <code>https://</code> , no trailing slash.
HTTPS pages can't call http://	Always put the <code>https://</code> Render URL in <code>config.js</code> .

Check before you call it deployed

- ☐ `/health` on the Render URL shows the right `chainId` and contract
- ☐ The frontend loads on a phone over mobile data
- ☐ A transaction from the live site appears on the explorer
- ☐ The AI step works on the live site (or deliberately runs in demo mode)
- ☐ Records are still listed after a redeploy (push any small commit and wait for Render to finish)
- ☐ UptimeRobot shows the monitor as **Up**

Security

A leaked private key is gone the moment it's pushed. Bots scan GitHub for keys around the clock and drain them within minutes, testnet or not.

Rule	Why / how
Burner wallets only	Make fresh MetaMask accounts for the hackathon. Never use a wallet that has held real funds.
.env never reaches GitHub	The starter's <code>.gitignore</code> already excludes it. Run <code>git status</code> before every push: if <code>.env</code> is listed, stop.
No secrets in frontend code	Everything in <code>frontend/</code> is public, including <code>config.js</code> . <code>PRIVATE_KEY</code> , <code>DATABASE_URL</code> (it holds your database password) and AI keys live only in <code>backend/.env</code> and on Render. The contract address and ABI are public anyway.
Secrets go in the host's dashboard	On Render, add environment variables in the service settings, never in the repo.
A leaked key must be replaced	Deleting the commit isn't enough: it stays in the history. Make a new wallet or key, move any tokens, update <code>.env</code> and Render.
Testnet only	If MetaMask ever shows a mainnet network while you're building, stop and switch back.
No personal data on-chain	The chain is public and permanent. Names, health records, ID numbers: off-chain, with only a hash on-chain.

Before your first push

```
git status # .env must NOT be listed
git check-ignore backend/.env # should print backend/.env
```

If you pushed a key

1. Create a new burner wallet in MetaMask and move any test tokens to it.
2. Put the new private key in `backend/.env` and on Render.
3. If an AI key leaked, revoke it in the provider's dashboard and create a new one. If `DATABASE_URL` leaked, reset the database password in Neon or Supabase.
4. Tell a mentor. Don't spend time rewriting git history during the hackathon.